

前端工程化_4小时速通

1. ES6

1.1. let

1.1.1. 越域

1.1.2. 重复声明

1.1.3. 变量提升

1.2. const

1.3. 解构

1.3.1. 数组解构

1.3.2. 对象解构

1.4. 链判断

1.5. 参数默认值

1.6. 箭头函数

1.7. 模板字符串

1.8. Promise

1.8.1. fetch api

1.8.2. Promise状态

1.8.3. Promise对象

1.9. Async 函数

1.10. 模块化

1.10.1. 工程架构

1.10.2. index.html

1.10.3. user.js

1.10.4. main.js

2. npm

2.1. 环境

2.2. 命令

2.3. 使用流程

3. Vite

3.1. 简介

3.2. 实战

3.2.1. 创建项目

3.2.2. 安装依赖

3.2.3. 项目启动

3.2.4. 项目打包

3.2.5. 项目部署

4. Vue3

4.1. 组件化

4.2. SFC

4.3. Vue工程

4.3.1. 创建&运行

4.3.2. 运行原理

4.4. 基础使用

4.4.1. 插值

4.4.2. 指令

4.4.2.1. 事件绑定: v-on

4.4.2.2. 条件判断: v-if

4.4.2.3. 循环: v-for

4.4.2.4. 更多指令

4.4.3. 属性绑定

4.4.4. 响应式 - ref()

4.4.4.1. 使用方式

4.4.4.2. 深层响应性

4.4.5. 响应式 - reactive()

4.4.6. 表单绑定

4.4.7. 计算属性 - computed

4.5. 进阶用法

4.5.1. 监听 - watch

4.5.2. 生命周期

4.5.3. 组件传值

4.5.3.1. 父传子 - Props

4.5.3.2. 子传父 - Emit

4.5.4. 插槽 - Slots

4.5.4.1. 基本使用

4.5.4.2. 默认内容

4.5.4.3. 具名插槽

4.6. 总结

5. Vue-Router

5.1. 理解路由

5.2. 路由入门

5.2.1. 项目创建

5.2.2. 整合vue-router

5.2.3. 运行流程

5.2.4. 路由配置

5.2.5. 路径参数

5.3. 嵌套路由

5.4. 编程式

5.4.1. useRoute: 路由数据

5.4.2. useRouter: 路由器

5.5. 路由传参

5.5.1. params 参数

5.5.2. query 参数

5.6. 导航守卫

5.7. 总结

6. Axios

6.1. 简介

6.2. 请求

6.2.1. get请求

6.2.2. post请求

6.3. 响应

6.4. 配置

6.5. 拦截器

7. Pinia

7.1. 核心

7.2. 整合

7.3. 案例

7.4. setup写法

8. 脚手架

9. Ant Design Vue

9.1. 整合

9.2. 布局

9.3. 组件

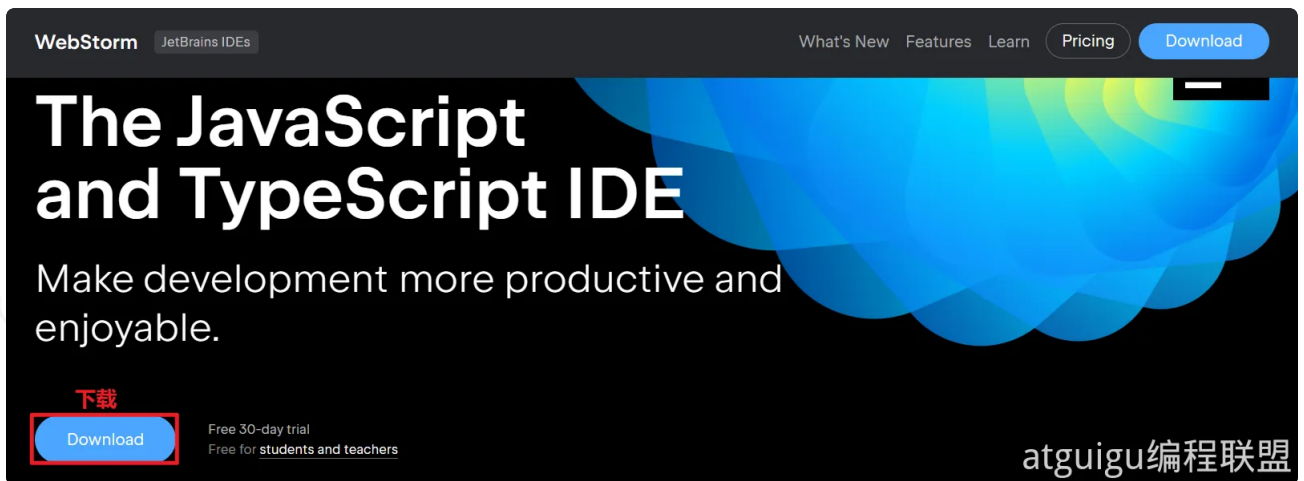
10. 接下来

B站视频地址：

- 尚硅谷：<https://www.bilibili.com/video/BV14t421M7CL>

前置准备：

- 默认你已经掌握：HTML、CSS、JS；才可以继续学习此课程
- 下载 `WebStorm` ,<https://www.jetbrains.com/webstorm/>；双击安装，下一步下一步即可



1. ES6

- ECMAScript (ES) 是规范、JavaScript 是 ES 的实现
- ES6 的第一个版本 在 2015 年 6 月发布，正式名称是《ECMAScript 2015 标准》（简称 ES2015）
- ES6 指是 5.1 版以后的 JavaScript 的下一代标准，涵盖了 ES2015、ES2016、ES2017 等等

1.1. let

推荐使用 `let` 关键字替代 `var` 关键字声明变量，因为 `var` 存在诸多问题，比如：

1.1.1. 越域

```
1 {  
2     var a = 1;  
3     let b = 2;  
4 }  
5 console.log(a); // 1  
6 console.log(b); // ReferenceError: b is not defined
```

1.1.2. 重复声明

```
1 // var 可以声明多次  
2 // let 只能声明一次  
3 var m = 1  
4 var m = 2  
5 let n = 3  
6 // let n = 4  
7 console.log(m) // 2  
8 console.log(n) // Identifier 'n' has already been declared
```

1.1.3. 变量提升

```
1 // var 会变量提升
2 // let 不存在变量提升
3 console.log(x); // undefined
4 var x = 10;
5 console.log(y); //ReferenceError: y is not defined
6 let y = 20;
```

1.2. const

```
1 // 1. 声明之后不允许改变
2 // 2. 一旦声明必须初始化, 否则会报错
3 const a = 1;
4 a = 3; //Uncaught TypeError: Assignment to constant variable.
```

1.3. 解构

1.3.1. 数组解构

```
1 let arr = [1, 2, 3];
2 //以前我们想获取其中的值, 只能通过角标。ES6 可以这样:
3 const [x, y, z] = arr; // x, y, z 将与 arr 中的每个位置对应来取值
4 // 然后打印
5 console.log(x, y, z);
```

1.3.2. 对象解构

```
1 const person = {
2   name: "jack",
3   age: 21,
4   language: ['java', 'js', 'css']
5 }
6 // 解构表达式获取值, 将 person 里面每一个属性和左边对应赋值
7 const {name, age, language} = person;
8 // 等价于下面
9 // const name = person.name;
10 // const age = person.age;
11 // const language = person.language;
12 // 可以分别打印
13 console.log(name);
14 console.log(age);
15 console.log(language);
16 // 扩展: 如果想要将 name 的值赋值给其他变量, 可以如下, nn 是新的变量名
17 const {name: nn, age, language} = person;
18 console.log(nn);
19 console.log(age);
20 console.log(language);
```

1.4. 链判断

如果读取对象内部的某个属性, 往往需要判断一下, 属性的上层对象是否存在。

比如, 读取`message.body.user.firstName`这个属性, 安全的写法是写成下面这样。

```
1 let message = null;
2 // 错误的写法
3 const firstName = message.body.user.firstName || 'default';
4
5 // 正确的写法
6 const firstName = (message
7   && message.body
8   && message.body.user
9   && message.body.user.firstName) || 'default';
10 console.log(firstName)
```

这样的层层判断非常麻烦，因此 ES2020 引入了“链判断运算符”（optional chaining operator）`?.`，简化上面的写法。

```
1 const firstName = message?.body?.user?.firstName || 'default';
```

1.5. 参数默认值

```
1 //在 ES6 以前，我们无法给一个函数参数设置默认值，只能采用变通写法：
2 function add(a, b) {
3   // 判断 b 是否为空，为空就给默认值 1
4   b = b || 1;
5   return a + b;
6 }
7 // 传一个参数
8 console.log(add(10));
9
10 //现在可以这么写：直接给参数写上默认值，没传就会自动使用默认值
11 function add2(a, b = 1) {
12   return a + b;
13 }
14
15 // 传一个参数
16 console.log(add2(10));
```

1.6. 箭头函数

```
1 //以前声明一个方法
2 // var print = function (obj) {
3 //   console.log(obj);
4 // }
5 // 可以简写为:
6 let print = obj => console.log(obj);
7 // 测试调用
8 print(100);
```

```
1 // 两个参数的情况:
2 let sum = function (a, b) {
3   return a + b;
4 }
5 // 简写为:
6 //当只有一行语句, 并且需要返回结果时, 可以省略 {}, 结果会自动返回。
7 let sum2 = (a, b) => a + b;
8 //测试调用
9 console.log(sum2(10, 10)); //20
10 // 代码不止一行, 可以用`{}`括起来
11 let sum3 = (a, b) => {
12   c = a + b;
13   return c;
14 };
15 //测试调用
16 console.log(sum3(10, 20)); //30
```

1.7. 模板字符串

```

1  let info = "你好，我的名字是： [" + name + " ]，年龄是： [" + age + " ]，邮箱是： [" +
2  console.log(info);
3
4  # 模板字符串的写法
5  let info = `你好，我的名字是： ${name}，年龄是： ${person.age}，邮箱是： ${person.em
6  console.log(info);

```

1.8. Promise

代表 异步对象，类似Java中的 `CompletableFuture`

Promise 是现代 JavaScript 中异步编程的基础，是一个由异步函数返回的可以向我们指示当前操作所处的状态的对象。在 Promise 返回给调用者的时候，操作往往还没有完成，但 Promise 对象可以让我们操作最终完成时对其进行处理（无论成功还是失败）

`fetch` 是浏览器支持从远程获取数据的一个函数，这个函数返回的就是 **Promise 对象**

```

1  const fetchPromise = fetch(
2    "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-s
3  );
4
5  console.log(fetchPromise);
6
7  fetchPromise.then((response) => {
8    console.log(`已收到响应： ${response.status}`);
9  });
10
11 console.log("已发送请求.....");
12

```

1.8.1. fetch api

fetch 是浏览器支持从远程获取数据的一个函数，这个函数返回的就是 **Promise 对象**

```
JavaScript |
1  const fetchPromise = fetch(
2    "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
3  );
4
5  console.log(fetchPromise);
6
7  fetchPromise.then((response) => {
8    console.log(`已收到响应: ${response.status}`);
9  });
10
11 console.log("已发送请求.....");
12
```

通过 fetch() API 得到一个 Response 对象；

- response.status: 读取响应状态码
- response.json(): 读取响应体json数据；（这也是个异步对象）

```
JavaScript |
1  const fetchPromise = fetch(
2    "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
3  );
4
5  fetchPromise.then((response) => {
6    const jsonPromise = response.json();
7    jsonPromise.then((json) => {
8      console.log(json[0].name);
9    });
10 });
11
```

1.8.2. Promise状态

首先，Promise 有三种状态：

- 待定 (pending)：初始状态，既没有被兑现，也没有被拒绝。这是调用 `fetch()` 返回 Promise 时的状态，此时请求还在进行中。
- 已兑现 (fulfilled)：意味着操作成功完成。当 Promise 完成时，它的 `then()` 处理函数被调用。
- 已拒绝 (rejected)：意味着操作失败。当一个 Promise 失败时，它的 `catch()` 处理函数被调用。

1.8.3. Promise对象

```
1 const promise = new Promise((resolve, reject) => {  
2   // 执行异步操作  
3   if (/* 异步操作成功 */) {  
4     resolve(value); // 调用 resolve, 代表 Promise 将返回成功的结果  
5   } else {  
6     reject(error); // 调用 reject, 代表 Promise 会返回失败结果  
7   }  
8 });
```

示例：


```
1  let get = function (url, data) {
2      return new Promise((resolve, reject) => {
3          $.ajax({
4              url: url,
5              type: "GET",
6              data: data,
7              success(result) {
8                  resolve(result);
9              },
10             error(error) {
11                 reject(error);
12             }
13         });
14     })
15 }
```

1.9. Async 函数

async function 声明创建一个绑定到给定名称的新异步函数。函数体内允许使用 **await** 关键字，这使得我们可以更简洁地编写基于 **promise** 的异步代码，并且避免了显式地配置 **promise** 链的需要。

- **async 函数** 是使用 **async** 关键字声明的函数。async 函数是 **AsyncFunction** 构造函数的实例，并且其中允许使用 **await** 关键字。
- **async** 和 **await** 关键字让我们可以用一种更简洁的方式写出基于 **Promise** 的异步行为，而无需刻意地链式调用 **promise**。
- **async 函数** 返回的还是 **Promise** 对象

```
1  async function myFunction() {
2      // 这是一个异步函数
3
4  }
```

在异步函数中，你可以在调用一个返回 Promise 的函数之前使用 **await** 关键字。这使得代码在该点上等待，直到 Promise 被完成，这时 Promise 的响应被当作返回值，或者被拒绝的响应被作为错误抛出。

```
1  async function fetchProducts() {
2    try {
3      // 在这一行之后，我们的函数将等待 `fetch()` 调用完成
4      // 调用 `fetch()` 将返回一个“响应”或抛出一个错误
5      const response = await fetch(
6        "https://mdn.github.io/learning-area/javascript/apis/fetching-data/can-store/products.json",
7      );
8      if (!response.ok) {
9        throw new Error(`HTTP 请求错误: ${response.status}`);
10     }
11     // 在这一行之后，我们的函数将等待 `response.json()` 的调用完成
12     // `response.json()` 调用将返回 JSON 对象或抛出一个错误
13     const json = await response.json();
14     console.log(json[0].name);
15   } catch (error) {
16     console.error(`无法获取产品列表: ${error}`);
17   }
18 }
19
20 fetchProducts();
```

1.10. 模块化

将 JavaScript 程序拆分为可按需导入的单独模块的机制。Node.js 已经提供这个能力很长时间了，还有很多的 JavaScript 库和框架已经开始了模块的使用（例如，[CommonJS](#) 和基于 [AMD](#) 的其他模块系统 如 [RequireJS](#)，以及最新的 [Webpack](#) 和 [Babel](#)）。

好消息是，最新的浏览器开始原生支持模块功能了。

1.10.1. 工程架构

demo-module
> libs
index.html
js main.js atguigu编程联盟

1.10.2. index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Title</title>
6     <script src="main.js" type="module"/>
7 </head>
8 <body>
9 <h1>模块化测试</h1>
10 </body>
11 </html>
```

1.10.3. user.js

放在 libs/user.js

```
1  const user = {
2      username: "张三",
3      age: 18
4  }
5
6  const isAdult = (age)=>{
7      if (age > 18){
8          console.log("成年人")
9      }else {
10         console.log("未成年")
11     }
12 }
13
14
15 export {user,isAdult}
16
17 // Java 怎么模块化;
18 // 1、 druid.jar
19 // 2、 import 导入类
20
21 // JS 模块化;
22 // 1、 xxx.js
23 // 2、 xxx.js 暴露功能;
24 // 3、 import 导入 xxx.js 的功能
25 //xxx.js 暴露的功能, 别人才能导入
26
```

1.10.4. main.js

```
1  // 所有的功能不用写在一个JS中
2  import {user,isAdult} from './libs/user.js'
3
4
5  alert("当前用户: "+user.username)
6
7  isAdult(user.age);
```

2. npm

npm 是 nodejs 中进行 **包管理** 的工具；

下载：<https://nodejs.org/en>

2.1. 环境

- 安装 Node.js
- 配置 npm

```
1  npm config set registry https://registry.npmirror.com #设置国内阿里云镜像源
2  npm config get registry #查看镜像源
```

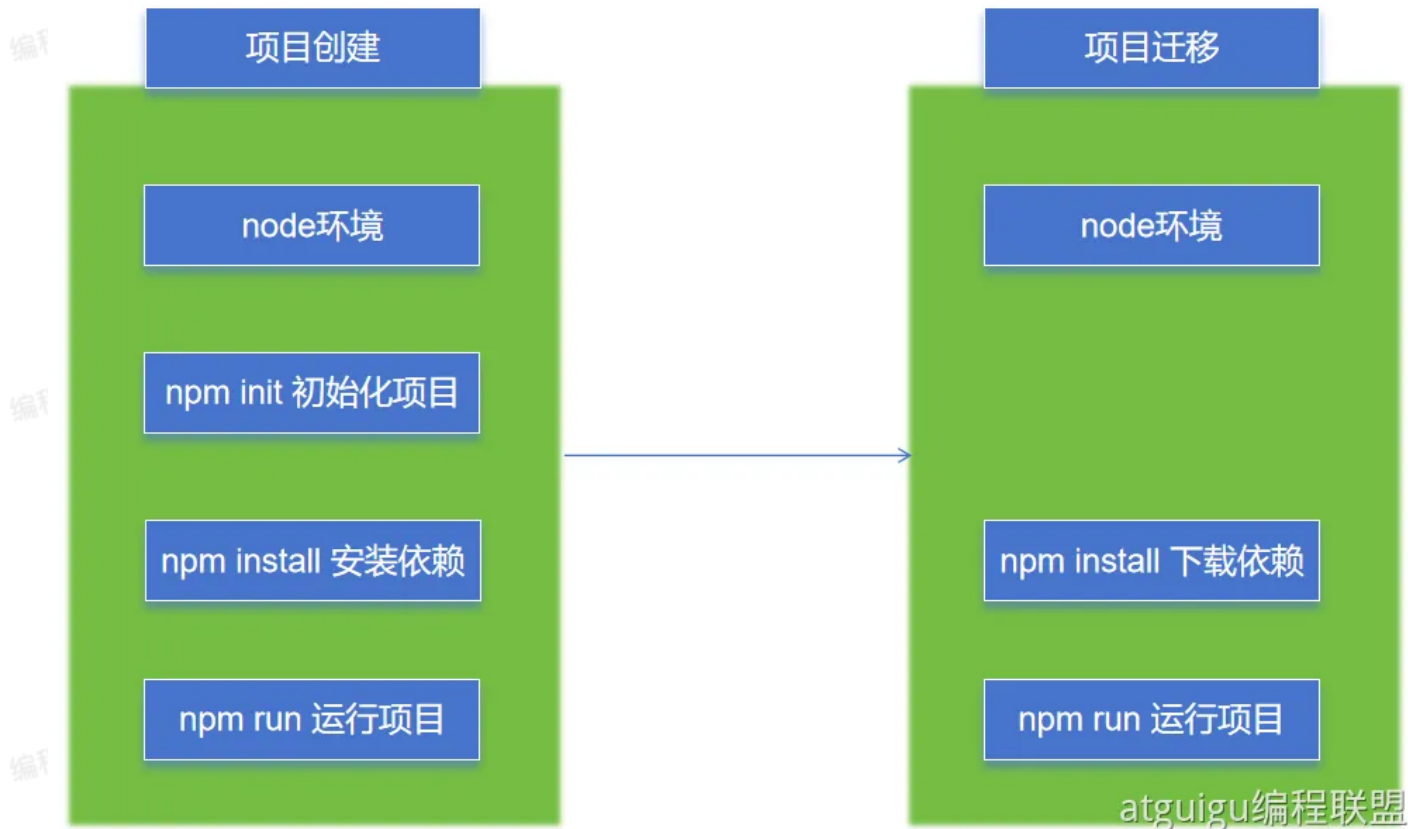
2.2. 命令

- npm init：项目初始化；
 - npm init -y：默认一路yes，不用挨个输入信息
- npm install 包名：安装js包到项目中（仅当前项目有效）。指定 **包名**，或者 **包名@版本号**
 - npm install -g：全局安装，所有都能用
 - 可以去 [npm仓库](#) 搜索第三方库
- npm update 包名：升级包到最新版本
- npm uninstall 包名：卸载包
- npm run：项目运行

2.3. 使用流程

Java：

- 项目创建：Java环境 ==》 maven 初始化 ==》 添加依赖 ==》 运行项目
- 项目迁移：Java环境 ==》 maven 下载依赖 ==》 运行项目



3. Vite

3.1. 简介

官网: <https://cn.vitejs.dev>

Vite

下一代的前端工具链

为开发提供极速响应

开始

为什么选 Vite?

在 GitHub 上查看

ViteConf 23!

- 快速创建前端项目脚手架
- 统一的工程化规范: 目录结构、代码规范、git提交规范 等



atguigu编程联盟

- 自动化构建和部署：前端脚手架可以自动进行代码打包、压缩、合并、编译等常见的构建工作，可以通过集成自动化部署脚本，自动将代码部署到测试、生产环境等；

3.2. 实战

3.2.1. 创建项目

```
1 npm create vite #根据向导选择技术栈
```

3.2.2. 安装依赖

```
1 npm install #安装项目所有依赖
2
3 npm install axios #安装指定依赖到当前项目
4 npm install -g xxx # 全局安装
```

3.2.3. 项目启动

```
1 npm run dev #启动项目
```

3.2.4. 项目打包

```
1 npm run build #构建后 生成 dist 文件夹
```

3.2.5. 项目部署

- 前后分离方式：需要把 dist 文件夹内容部署到如 nginx 之类的服务器上。
- 前后不分离方式：把 dist 文件夹内容复制到 SpringBoot 项目 `resources` 下面

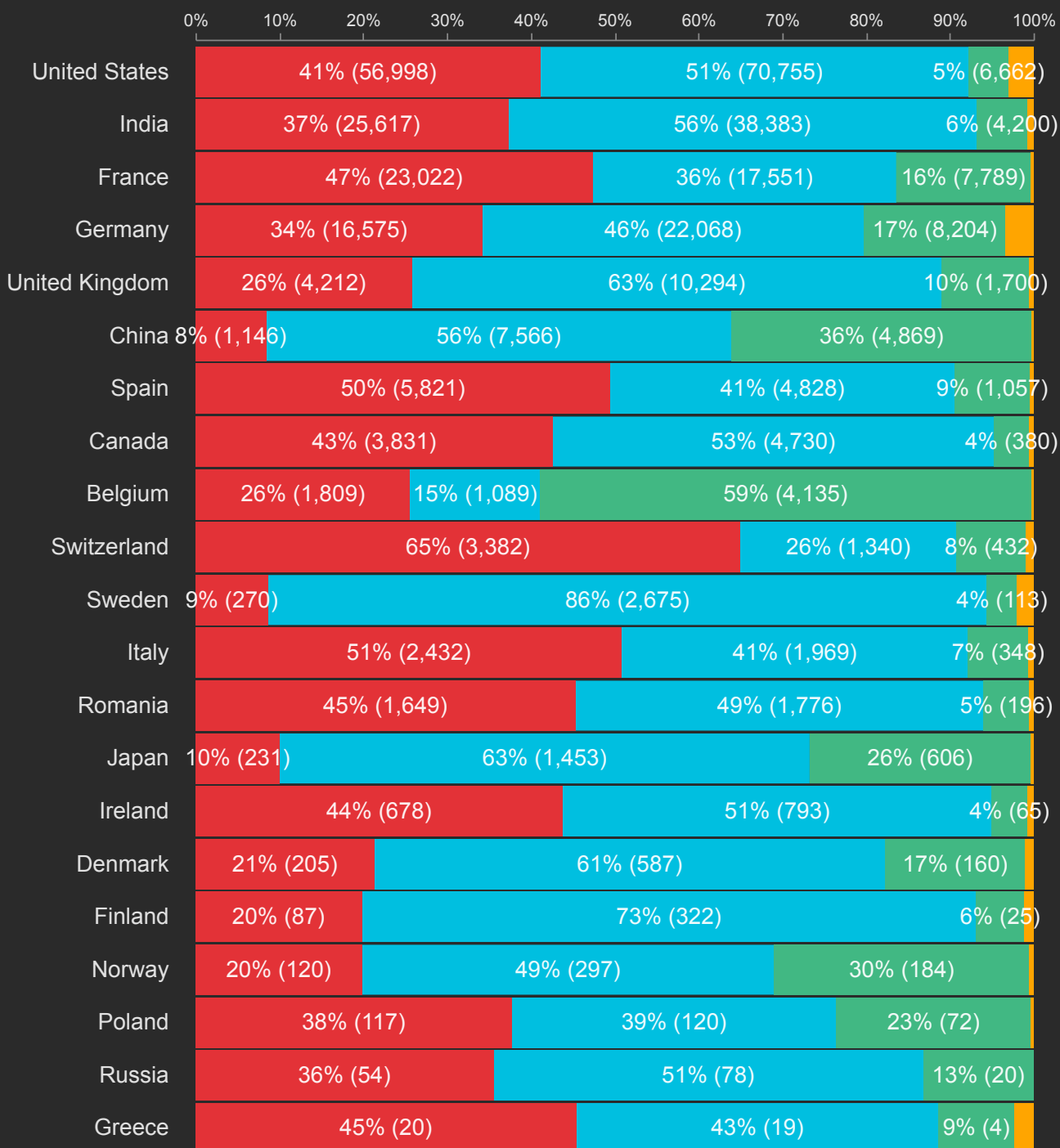
4. Vue3

2023 年前端框架各个国家使用占比。

Frontend framework

● Angular ● Vue ● React
● Others

Percentage of jobs by Country in 2023



devjobs scanner

我们使用 vite 创建 vue项目脚手架，并测试Vue功能

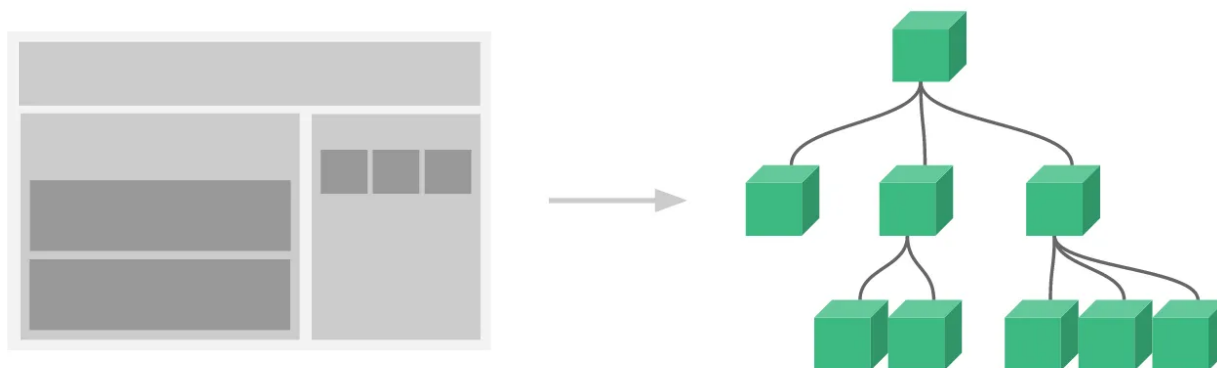
```
1 npm create vite #根据向导选择技术栈
```

4.1. 组件化

组件系统是一个抽象的概念：

- 组件：小型、独立、可复用的单元
- 组合：通过组件之间的组合、包含关系构建出一个完整应用

几乎任意类型的应用界面都可以抽象为一个组件树：



atguigu编程联盟

4.2. SFC

Vue 的**单文件组件** (即 *.vue 文件，英文 Single-File Component，简称 **SFC**) 是一种特殊的文件格式，使我们能够将一个 Vue 组件的模板、逻辑与样式封装在单个文件中。

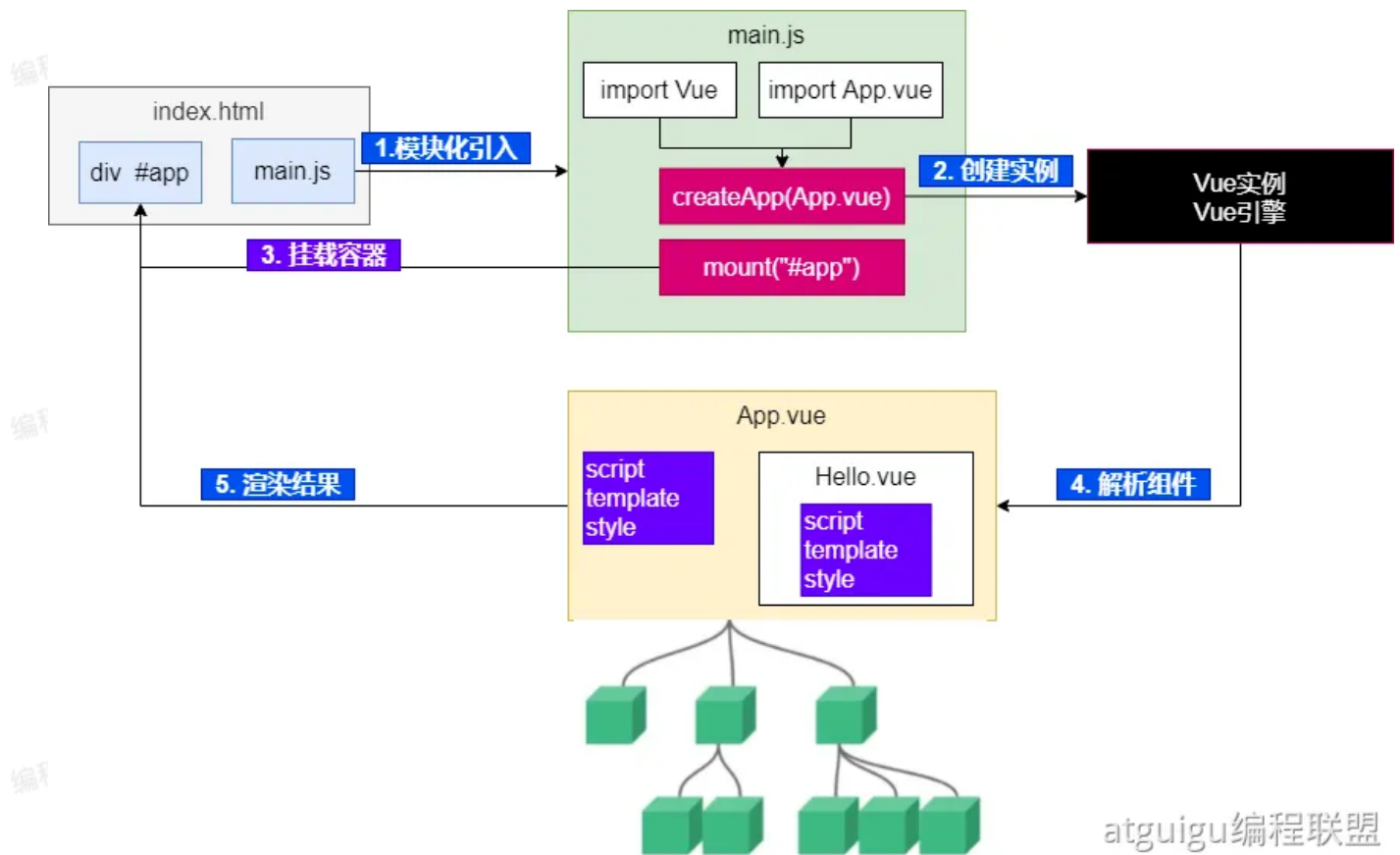
```
1 <script setup>
2   //编写脚本
3 </script>
4
5 <template>
6   //编写页面模板
7 </template>
8
9 <style scoped>
10  //编写样式
11 </style>
```

4.3. Vue工程

4.3.1. 创建&运行

```
1 npm create vite # 按照提示选择Vue
2 npm run dev #项目运行命令
```

4.3.2. 运行原理



atguigu编程联盟

4.4. 基础使用

4.4.1. 插值

```
1 <script setup>
2 //基本数据
3 let name = "张三"
4 let age = 18
5
6 //对象数据
7 let car = {
8   brand: "奔驰",
9   price: 777
10 }
11 </script>
12
13 <template>
14   <p> name: {{name}} </p>
15   <p> age: {{age}} </p>
16   <div style="border: 3px solid red">
17     <p>品牌: {{car.brand}}</p>
18     <p>价格: {{car.price}}</p>
19   </div>
20 </template>
21
22 <style scoped>
23
24 </style>vue
```

4.4.2. 指令

4.4.2.1. 事件绑定: v-on

使用 `v-on` 指令, 可以为元素绑定事件。可以简写为 `@`

```

1 <script setup>
2 //定义事件回调
3 function buy(){
4   alert("购买成功");
5 }
6 </script>
7 <template>
8   <button v-on:click="buy">购买</button>
9   <button @click="buy">购买</button>
10 </template>
11 <style scoped>
12 </style>

```

v-on:submit.prevent="onSubmit"

Name

Starts with v-
May be omitted when
using shorthands

Argument

Follows the colon
or shorthand
symbol

Modifiers

Denoted by the
leading dot

Value

Interpreted as
JavaScript expressions

atguigu编程联盟

Modifiers: 修饰符详情

<https://cn.vuejs.org/guide/essentials/event-handling.html>

4.4.2.2. 条件判断: v-if

4.4.2.3. 循环: v-for

4.4.2.4. 更多指令

<https://cn.vuejs.org/api/built-in-directives.html>; 后期我们都会用到。

v-text

v-html

v-show

v-if

v-else

v-else-if

v-for

v-on

v-bind

v-model

v-slot

v-pre

v-once

v-memo

v-cloak

4.4.3. 属性绑定

- 使用 `v-bind:属性='xx'` 语法，可以为标签的某个属性绑定值；
- 可以简写为 `:属性='xx'`

```
1 <script setup>
2   //
3   let url = "http://www.baidu.com"
4 </script>
5
6 <template>
7   <a v-bind:href=url>go</a>
8   <a :href=url>go</a>
9 </template>
10
11 <style scoped>
12
13 </style>
14
```

注意：此时如果我们想修改变量的值，属性值并不会跟着修改。可以测试下。因为还没有响应式特性

4.4.4. 响应式 – ref()

数据的动态变化需要反馈到页面；

Vue通过 `ref()` 和 `reactive()` 包装数据，将会生成一个数据的代理对象。vue内部的 基于依赖追踪的响应式系统 就会追踪感知数据变化，并触发页面的重新渲染。

4.4.4.1. 使用方式

使用步骤：

1. 使用 `ref()` 包装原始类型、对象类型数据，生成代理对象
2. 任何方法、js代码中，使用 `代理对象.value` 的形式读取和修改值
3. 页面组件中，直接使用 代理对象

注意：推荐使用 `const`（常量）声明代理对象。代表代理对象不可变，但是内部值变化会被追踪。

```
1 <script setup>
2 import { ref } from 'vue'
3
4 const count = ref(0)
5
6 function increment() {
7   count.value++
8 }
9 </script>
10
11 <template>
12   <button @click="increment">
13     {{ count }}
14   </button>
15 </template>
```

4.4.4.2. 深层响应性

```
1 import { ref } from 'vue'
2
3 const obj = ref({
4   nested: { count: 0 },
5   arr: ['foo', 'bar']
6 })
7
8 function mutateDeeply() {
9   // 以下都会按照期望工作
10   obj.value.nested.count++
11   obj.value.arr.push('baz')
12 }
```

4.4.5. 响应式 – reactive()

使用步骤：

1. 使用 `reactive()` 包装对象类型数据，生成 代理对象
2. 任何方法、js代码中，使用 `代理对象.属性` 的形式读取和修改值
3. 页面组件中，直接使用 `代理对象.属性`

```
1 import { reactive } from 'vue'
2
3 const state = reactive({ count: 0 })
4
5 <button @click="state.count++">
6   {{ state.count }}
7 </button>
```

基本类型用 `ref()`，对象类型用 `reactive()`，`ref` 要用 `.value`，`reactive` 直接 `.`。页面取值永不变。

也可以 `ref` 一把梭，大不了 天天 `.value`

4.4.6. 表单绑定

复制如下模板到组件，编写你的代码，实现表单数据绑定。

```
1 <div style="display: flex;">
2   <div style="border: 1px solid black; width: 300px">
3     <form>
4       <h1>表单绑定</h1>
5       <p style="background-color: azure"><label>姓名(文本框): </label><input
6         /></p>
7       <p style="background-color: azure"><label>同意协议(checkbox): </label>
8         <input type="checkbox"/>
9       <p style="background-color: azure">
10        <label>兴趣(多选框): </label><br/>
11        <label><input type="checkbox" value="足球"/>足球</label>
12        <label><input type="checkbox" value="篮球"/>篮球</label>
13        <label><input type="checkbox" value="羽毛球"/>羽毛球</label>
14        <label><input type="checkbox" value="乒乓球"/>乒乓球</label>
15      </p>
16      <p style="background-color: azure">
17        <label>性别(单选框): </label>
18        <label><input type="radio" name="sex" value="男">男</label>
19        <label><input type="radio" name="sex" value="女">女</label>
20      </p>
21      <p style="background-color: azure">
22        <label>学历(单选下拉列表): </label>
23        <select>
24          <option disabled value="">选择学历</option>
25          <option>小学</option>
26          <option>初中</option>
27          <option>高中</option>
28          <option>大学</option>
29        </select>
30      </p>
31      <p style="background-color: azure">
32        <label>课程(多选下拉列表): </label>
33        <br/>
34        <select multiple>
35          <option disabled value="">选择课程</option>
36          <option>语文</option>
37          <option>数学</option>
38          <option>英语</option>
39          <option>道法</option>
40        </select>
41      </p>
42    </form>
43  </div>
```

```
44 <div style="border: 1px solid blue;width: 200px">
45   <h1>结果预览</h1>
46   <p style="background-color: azure"><label>姓名: </label></p>
47   <p style="background-color: azure"><label>同意协议: </label>
48   </p>
49   <p style="background-color: azure">
50     <label>兴趣: </label>
51   </p>
52   <p style="background-color: azure">
53     <label>性别: </label>
54   </p>
55   <p style="background-color: azure">
56     <label>学历: </label>
57   </p>
58   <p style="background-color: azure">
59     <label>课程: </label>
60   </p>
61 </div>
62 </div>
```

4.4.7. 计算属性 – computed

计算属性：根据已有数据计算出新数据

```
1 <script setup>
2 import {computed, reactive, toRef, toRefs} from "vue";
3
4 //省略基础代码
5
6 const totalPrice = computed(()=>{
7   return price.value * num.value - coupon.value*100
8 })
9
10 </script>
11
12 <template>
13   <div class="car">
14     <h2>优惠券: {{ car.coupon }} 张</h2>
15     <h2>数量: {{ car.num }}</h2>
16     <h2>单价: {{ car.price }}</h2>
17     <h1>总价: {{totalPrice}}</h1>
18     <button @click="getCoupon">获取优惠</button>
19     <button @click="addNum">加量</button>
20     <button @click="changePrice">加价</button>
21   </div>
22
23 </template>
24
25 <style scoped>
26
27 </style>
```

4.5. 进阶用法

4.5.1. 监听 – watch

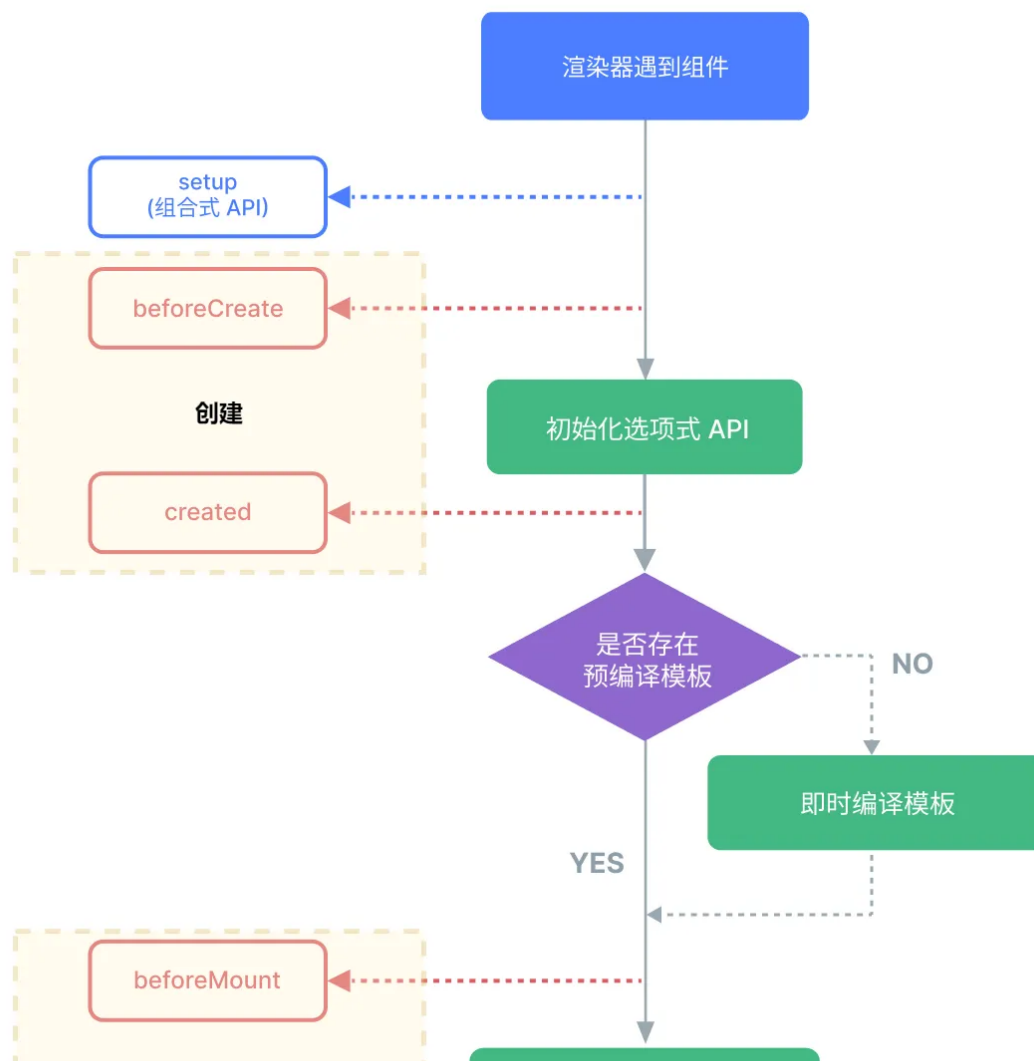
```

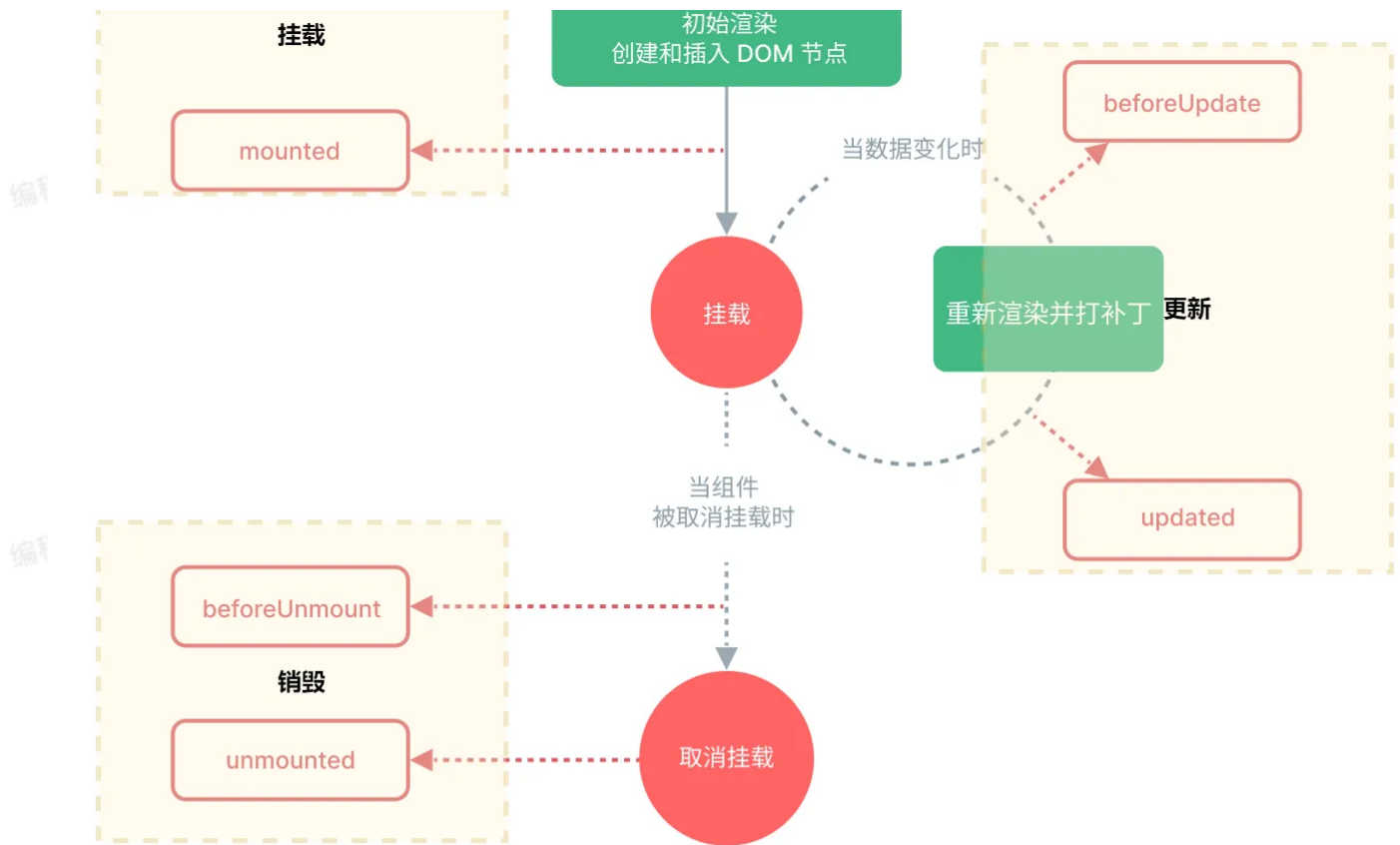
1 watch(num, (value, oldValue, onCleanup) => {
2   console.log("newValue: " + value + "; oldValue:" + oldValue)
3   onCleanup(() => {
4     console.log("onCleanup....")
5   })
6 })
7

```

4.5.2. 生命周期

每个 Vue 组件实例在创建时都需要经历一系列的初始化步骤，比如设置好数据侦听，编译模板，挂载实例到 DOM，以及在数据改变时更新 DOM。在此过程中，它也会运行被称为生命周期钩子的函数，让开发者有机会在特定阶段运行自己的代码。





atguigu编程联盟

生命周期整体分为四个阶段，分别是：创建、挂载、更新、销毁，每个阶段都有两个钩子，一前一后。

常用的钩子：

- onMounted(挂载完毕)
- onUpdated(更新完毕)
- onBeforeUnmount(卸载之前)

```
1 <script setup>
2 import {onBeforeMount, onBeforeUpdate, onMounted, onUpdated, ref} from "vue";
3 const count = ref(0)
4 const btn01 = ref()
5
6 // 生命周期钩子
7 onBeforeMount(()=>{
8   console.log('挂载之前', count.value, document.getElementById("btn01"))
9 })
10 onMounted(()=>{
11   console.log('挂载完毕', count.value, document.getElementById("btn01"))
12 })
13 onBeforeUpdate(()=>{
14   console.log('更新之前', count.value, btn01.value.innerHTML)
15 })
16 onUpdated(()=>{
17   console.log('更新完毕', count.value, btn01.value.innerHTML)
18 })
19 </script>
20
21 <template>
22   <button ref="btn01" @click="count++"> {{count}} </button>
23 </template>
24
25 <style scoped>
26 </style>
27
```

4.5.3. 组件传值

4.5.3.1. 父传子 – Props

父组件给子组件传递值；

单向数据流效果：

- 父组件修改值，子组件发生变化
- 子组件修改值，父组件不会感知到


```

1 //父组件给子组件传递数据：使用属性绑定
2 <Son :books="data.books" :money="data.money"/>
3
4 //子组件定义接受父组件的属性
5 let props = defineProps({
6   money: {
7     type: Number,
8     required: true,
9     default: 200
10  },
11  books: Array
12 });

```

4.5.3.2. 子传父 – Emit

props 用来父传子，emit 用来子传父

```

1 //子组件定义发生的事件
2 let emits = defineEmits(['buy']);
3 function buy(){
4   // props.money -= 5;
5   emits('buy', -5);
6 }
7
8 //父组件感知事件和接受事件值
9 <Son :books="data.books" :money="data.money"
10   @buy="moneyMinis"/>

```

思考：

- 利用父子传值即可

4.5.4. 插槽 – Slots

子组件可以使用插槽接受模板内容。

4.5.4.1. 基本使用

```
1 <!-- 组件定义 -->
2 <button class="fancy-btn">
3   <slot></slot> <!-- 插槽出口 -->
4 </button>
5
6 <!-- 组件使用 -->
7 <FancyButton>
8   Click me! <!-- 插槽内容 -->
9 </FancyButton>
```

4.5.4.2. 默认内容

```
1 <button type="submit">
2   <slot>
3     Submit <!-- 默认内容 -->
4   </slot>
5 </button>
```

4.5.4.3. 具名插槽

定义

```
1 <div class="container">
2   <header>
3     <slot name="header"></slot>
4   </header>
5   <main>
6     <slot></slot>
7   </main>
8   <footer>
9     <slot name="footer"></slot>
10  </footer>
11 </div>
```

使用: `v-slot` 可以简写为 `#`

```
1 <BaseLayout>
2   <template v-slot:header>
3     <!-- header 插槽的内容放这里 -->
4   </template>
5 </BaseLayout>
```

4.6. 总结

插值: `{{ }}`

指令

`v-html`

`v-text`

`v-if`

`v-else`

`v-for`

`v-on`

`v-bind`

`v-model`

`v-slot`

内置函数

`ref()`

`reactive()`

`computed()`

`watch()`

`watchEffect()`

`onBeforeMount()`

`onMounted()`

`onBeforeUpdate()`

`onUpdated()`

`defineProps()`

`defineEmits()`

atguigu编程联盟

几个简写:

- `v-on` = `@`

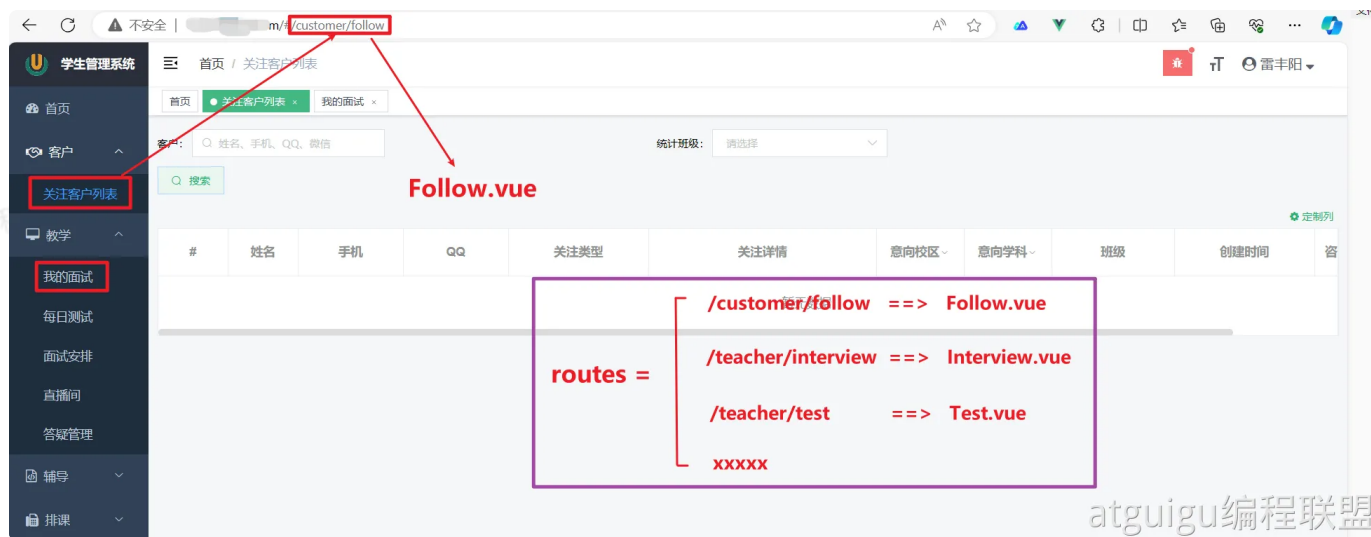
- `v-bind` = `:`

- `v-slot = #`

5. Vue-Router

5.1. 理解路由

前端系统根据页面路径，跳转到指定组件，展示出指定效果



5.2. 路由入门

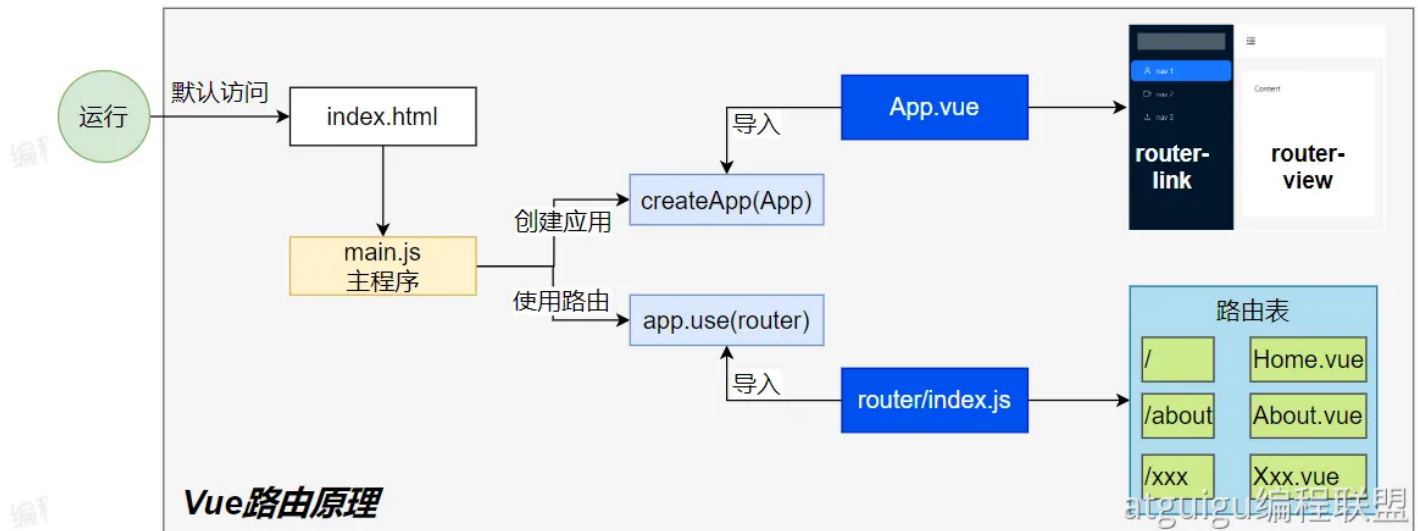
5.2.1. 项目创建

```
1 npm create vite
```

5.2.2. 整合vue-router

参照官网即可

5.2.3. 运行流程



5.2.4. 路由配置

编写一个新的组件，测试路由功能；步骤如下：

1. 编写 `router/index.js` 文件
2. 配置路由表信息
3. 创建路由器并导出
4. 在 `main.js` 中使用路由
5. 在页面使用 `router-link` `router-view` 完成路由功能

5.2.5. 路径参数

使用 `:变量名` 接受动态参数；这个成为 **路径参数**

```

1 const routes = [
2   // 匹配 /o/3549
3   { path: '/o/:orderId' },
4   // 匹配 /p/books
5   { path: '/p/:productName' },
6 ]

```

5.3. 嵌套路由

欢迎你：张三

[我的信息](#) [我的邮箱](#)



JavaScript

```
1  const routes = [  
2    {  
3      path: '/user/:id',  
4      component: User,  
5      children: [  
6        {  
7          // 当 /user/:id/profile 匹配成功  
8          // UserProfile 将被渲染到 User 的 <router-view> 内部  
9          path: 'profile',  
10         component: UserProfile,  
11       },  
12       {  
13         // 当 /user/:id/posts 匹配成功  
14         // UserPosts 将被渲染到 User 的 <router-view> 内部  
15         path: 'posts',  
16         component: UserPosts,  
17       },  
18     ],  
19   },  
20 ]
```

5.4. 编程式

5.4.1. useRouter：路由数据

路由传参跳转到指定页面后，页面需要取到传递过来的值，可以使用 `useRoute` 方法；
拿到当前页路由数据；可以做

1. 获取到当前路径
2. 获取到组件名
3. 获取到参数
4. 获取到查询字符串

```
1 import {useRoute} from 'vue-router'
2 const route = useRoute()
3 // 打印query参数
4 console.log(route.query)
5 // 打印params参数
6 console.log(route.params)
```

5.4.2. useRouter: 路由器

拿到路由器；可以控制跳转、回退等。

```
1 import {useRoute, useRouter} from "vue-router";
2
3 const router = useRouter();
4
5 // 字符串路径
6 router.push('/users/eduardo')
7
8 // 带有路径的对象
9 router.push({ path: '/users/eduardo' })
10
11 // 命名的路由，并加上参数，让路由建立 url
12 router.push({ name: 'user', params: { username: 'eduardo' } })
13
14 // 带查询参数，结果是 /register?plan=private
15 router.push({ path: '/register', query: { plan: 'private' } })
16
17 // 带 hash，结果是 /about#team
18 router.push({ path: '/about', hash: '#team' })
19
20 //注意：`params` 不能与 `path` 一起使用
21 router.push({ path: '/user', params: { username } }) //错误用法 -> /user
```

5.5. 路由传参

5.5.1. params 参数

```
1  <!-- 跳转并携带params参数 (to的字符串写法) -->
2  <RouterLink :to="/news/detail/001/新闻001/内容001">{{news.title}}</Router
   Link>
3
4  <!-- 跳转并携带params参数 (to的对象写法) -->
5  <RouterLink
6    :to="{
7      name:'xiang', //用name跳转, params情况下, 不可用path
8      params:{
9        id:news.id,
10       title:news.title,
11       content:news.title
12     }
13   }"
14  >
15   {{news.title}}
16 </RouterLink>
```

5.5.2. query 参数


```
1 <!-- 跳转并携带query参数（to的字符串写法） -->
2 <router-link to="/news/detail?a=1&b=2&content=欢迎你">
3   跳转
4 </router-link>
5
6 <!-- 跳转并携带query参数（to的对象写法） -->
7 <RouterLink
8   :to="{
9     //name:'xiang', //用name也可以跳转
10    path:'/news/detail',
11    query:{
12      id:news.id,
13      title:news.title,
14      content:news.content
15    }
16  }"
17 >
18   {{news.title}}
19 </RouterLink>
```

5.6. 导航守卫

我们只演示全局前置守卫。后置钩子等内容参照官方文档

```
1 import {createRouter, createWebHistory} from 'vue-router'
2 import HomeView from '../views/HomeView.vue'
3
4 const router = createRouter({
5   history: createWebHistory(import.meta.env.BASE_URL),
6   routes: [
7     {
8       path: '/',
9       name: 'home',
10      component: HomeView
11    },
12    {
13      path: '/about',
14      name: 'about',
15      // route level code-splitting
16      // this generates a separate chunk (About.[hash].js) for this
17      // route
18      // which is lazy-loaded when the route is visited.
19      component: () => import('../views/AboutView.vue')
20    },
21    {
22      path: '/user/:name',
23      name: 'User',
24      component: () => import('@views/user/UserInfo.vue'),
25      children: [
26        {
27          path: 'profile',
28          component: () => import('@views/user/Profile.vue')
29        },
30        {
31          path: 'posts',
32          component: () => import('@views/user/Posts.vue')
33        }
34      ]
35    }
36  ])
37
38 router.beforeEach(async (to, from) => {
39   console.log("守卫: to: ", to)
40   console.log("守卫: from: ", from)
41   if (to.fullPath === '/about') {
42     return "/"
43   }
44 })
```

5.7. 总结

路由配置

routes: 路由表

createRouter(): 创建路由

标签

router-link

router-view

函数

useRoute()

path

params

query

name

useRouter()

push

go

导航守卫

`router.beforeEach( async (to, from) => { })`

6. Axios

6.1. 简介

Axios 是一个基于 promise 的网络请求库，可以用于浏览器和 node.js

```
1 npm install axios
```

```
1 import axios from "axios"
2 axios.get('/user')
3   .then(res => console.log(res.data))
```

API测试服务器: <http://43.139.239.29/>

6.2. 请求

6.2.1. get请求

```
1 // 向给定ID的用户发起请求
2 axios.get('/user?ID=12345')
3   .then(function (response) {
4     // 处理成功情况
5     console.log(response);
6   })
7   .catch(function (error) {
8     // 处理错误情况
9     console.log(error);
10  })
11  .finally(function () {
12    // 总是会执行
13  });
```

携带请求参数

```
1 // 上述请求也可以按以下方式完成 (可选)
2 axios.get('/user', {
3   params: {
4     ID: 12345
5   }
6 })
7 .then(function (response) {
8   console.log(response);
9 })
10 .catch(function (error) {
11   console.log(error);
12 })
13 .finally(function () {
14   // 总是会执行
15 });
```

6.2.2. post请求

默认 `post` 请求体 中的数据将会以 `json` 方式提交

```
1 axios.post('/user', {
2   firstName: 'Fred',
3   lastName: 'Flintstone'
4 })
5 .then(function (response) {
6   console.log(response);
7 })
8 .catch(function (error) {
9   console.log(error);
10 });
```

6.3. 响应

响应的数据结构如下：

```
1 {  
2   // `data` 由服务器提供的响应  
3   data: {},  
4  
5   // `status` 来自服务器响应的 HTTP 状态码  
6   status: 200,  
7  
8   // `statusText` 来自服务器响应的 HTTP 状态信息  
9   statusText: 'OK',  
10  
11  // `headers` 是服务器响应头  
12  // 所有的 header 名称都是小写，而且可以使用方括号语法访问  
13  // 例如: `response.headers['content-type']`  
14  headers: {},  
15  
16  // `config` 是 `axios` 请求的配置信息  
17  config: {},  
18  
19  // `request` 是生成此响应的请求  
20  // 在node.js中它是最后一个ClientRequest实例 (in redirects),  
21  // 在浏览器中则是 XMLHttpRequest 实例  
22  request: {}  
23 }
```

6.4. 配置

```
1 const instance = axios.create({
2   baseURL: 'https://some-domain.com/api/',
3   timeout: 1000,
4   headers: {'X-Custom-Header': 'foobar'}
5 });
6
7 //可用的配置项如下:
8 {
9   // `url` 是用于请求的服务器 URL
10  url: '/user',
11
12  // `method` 是创建请求时使用的方法
13  method: 'get', // 默认值
14
15  // `baseURL` 将自动加在 `url` 前面, 除非 `url` 是一个绝对 URL。
16  // 它可以通过设置一个 `baseURL` 便于为 axios 实例的方法传递相对 URL
17  baseURL: 'https://some-domain.com/api/',
18
19  // `transformRequest` 允许在向服务器发送前, 修改请求数据
20  // 它只能用于 'PUT', 'POST' 和 'PATCH' 这几个请求方法
21  // 数组中最后一个函数必须返回一个字符串, 一个Buffer实例, ArrayBuffer, FormData, 或 Stream
22  // 你可以修改请求头。
23  transformRequest: [function (data, headers) {
24    // 对发送的 data 进行任意转换处理
25
26    return data;
27  }],
28
29  // `transformResponse` 在传递给 then/catch 前, 允许修改响应数据
30  transformResponse: [function (data) {
31    // 对接收的 data 进行任意转换处理
32
33    return data;
34  }],
35
36  // 自定义请求头
37  headers: {'X-Requested-With': 'XMLHttpRequest'},
38
39  // `params` 是与请求一起发送的 URL 参数
40  // 必须是一个简单对象或 URLSearchParams 对象
41  params: {
42    ID: 12345
43  },
44
```

```

45 // `paramsSerializer` 是可选方法，主要用于序列化 `params`
46 // (e.g. https://www.npmjs.com/package/qs, http://api.jquery.com/jquer
y.param/)
47 paramsSerializer: function (params) {
48   return Qs.stringify(params, {arrayFormat: 'brackets'})
49 },
50
51 // `data` 是作为请求体被发送的数据
52 // 仅适用 'PUT', 'POST', 'DELETE 和 'PATCH' 请求方法
53 // 在没有设置 `transformRequest` 时，则必须是以下类型之一：
54 // - string, plain object, ArrayBuffer, ArrayBufferView, URLSearchParams
55 // - 浏览器专属: FormData, File, Blob
56 // - Node 专属: Stream, Buffer
57 data: {
58   firstName: 'Fred'
59 },
60
61 // 发送请求体数据的可选语法
62 // 请求方式 post
63 // 只有 value 会被发送, key 则不会
64 data: 'Country=Brasil&City=Belo Horizonte',
65
66 // `timeout` 指定请求超时的毫秒数。
67 // 如果请求时间超过 `timeout` 的值，则请求会被中断
68 timeout: 1000, // 默认值是 `0` (永不超时)
69
70 // `withCredentials` 表示跨域请求时是否需要使用凭证
71 withCredentials: false, // default
72
73 // `adapter` 允许自定义处理请求，这使测试更加容易。
74 // 返回一个 promise 并提供一个有效的响应 (参见 lib/adapters/README.md)。
75 adapter: function (config) {
76   /* ... */
77 },
78
79 // `auth` HTTP Basic Auth
80 auth: {
81   username: 'janedoe',
82   password: 's00pers3cret'
83 },
84
85 // `responseType` 表示浏览器将要响应的数据类型
86 // 选项包括: 'arraybuffer', 'document', 'json', 'text', 'stream'
87 // 浏览器专属: 'blob'
88 responseType: 'json', // 默认值
89
90 // `responseEncoding` 表示用于解码响应的编码 (Node.js 专属)

```



```

91 // 注意: 忽略 `responseType` 的值为 'stream', 或者是客户端请求
92 // Note: Ignored for `responseType` of 'stream' or client-side requests
93 responseEncoding: 'utf8', // 默认值
94
95 // `xsrCookieName` 是 xsrf token 的值, 被用作 cookie 的名称
96 xsrfCookieName: 'XSRF-TOKEN', // 默认值
97
98 // `xsrHeaderName` 是带有 xsrf token 值的http 请求头名称
99 xsrfHeaderName: 'X-XSRF-TOKEN', // 默认值
100
101 // `onUploadProgress` 允许为上传处理进度事件
102 // 浏览器专属
103 onUploadProgress: function (progressEvent) {
104     // 处理原生进度事件
105 },
106
107 // `onDownloadProgress` 允许为下载处理进度事件
108 // 浏览器专属
109 onDownloadProgress: function (progressEvent) {
110     // 处理原生进度事件
111 },
112
113 // `maxContentLength` 定义了node.js中允许的HTTP响应内容的最大字节数
114 maxContentLength: 2000,
115
116 // `maxBodyLength` (仅Node) 定义允许的http请求内容的最大字节数
117 maxBodyLength: 2000,
118
119 // `validateStatus` 定义了对于给定的 HTTP状态码是 resolve 还是 reject promise。
120 // 如果 `validateStatus` 返回 `true` (或者设置为 `null` 或 `undefined`),
121 // 则promise 将会 resolved, 否则是 rejected。
122 validateStatus: function (status) {
123     return status >= 200 && status < 300; // 默认值
124 },
125
126 // `maxRedirects` 定义了要在node.js中要遵循的最大重定向数。
127 // 如果设置为0, 则不会进行重定向
128 maxRedirects: 5, // 默认值
129
130 // `socketPath` 定义了要在node.js中使用的UNIX套接字。
131 // e.g. '/var/run/docker.sock' 发送请求到 docker 守护进程。
132 // 只能指定 `socketPath` 或 `proxy` 。
133 // 若都指定, 这使用 `socketPath` 。
134 socketPath: null, // default
135
136 // `httpAgent` and `httpsAgent` define a custom agent to be used when performing http

```

```

137 // and https requests, respectively, in node.js. This allows options to
138 // be added like
139 // `keepAlive` that are not enabled by default.
140 httpAgent: new http.Agent({ keepAlive: true }),
141 httpsAgent: new https.Agent({ keepAlive: true }),
142
143 // `proxy` 定义了代理服务器的主机名, 端口和协议。
144 // 您可以使用常规的`http_proxy` 和 `https_proxy` 环境变量。
145 // 使用 `false` 可以禁用代理功能, 同时环境变量也会被忽略。
146 // `auth` 表示应使用HTTP Basic auth连接到代理, 并且提供凭据。
147 // 这将设置一个 `Proxy-Authorization` 请求头, 它会覆盖 `headers` 中已存在的自
148 // 定义 `Proxy-Authorization` 请求头。
149 // 如果代理服务器使用 HTTPS, 则必须设置 protocol 为`https`
150 proxy: {
151   protocol: 'https',
152   host: '127.0.0.1',
153   port: 9000,
154   auth: {
155     username: 'mikeymike',
156     password: 'rapunz3l'
157   }
158 },
159
160 // see https://axios-http.com/zh/docs/cancellation
161 cancelToken: new CancelToken(function (cancel) {
162   // `decompress` indicates whether or not the response body should be de
163   // compressed
164   // automatically. If set to `true` will also remove the 'content-encodi
165   // ng' header
166   // from the responses objects of all decompressed responses
167   // - Node only (XHR cannot turn off decompression)
168   decompress: true // 默认值
169 }

```

6.5. 拦截器

```
1 // 添加请求拦截器
2 axios.interceptors.request.use(function (config) {
3     // 在发送请求之前做点什么
4     return config;
5 }, function (error) {
6     // 对请求错误做点什么
7     return Promise.reject(error);
8 });
9
10 // 添加响应拦截器
11 axios.interceptors.response.use(function (response) {
12     // 2xx 范围内的状态码都会触发该函数。
13     // 对响应数据做点什么
14     return response;
15 }, function (error) {
16     // 超出 2xx 范围的状态码都会触发该函数。
17     // 对响应错误做点什么
18     return Promise.reject(error);
19 });
```

7. Pinia

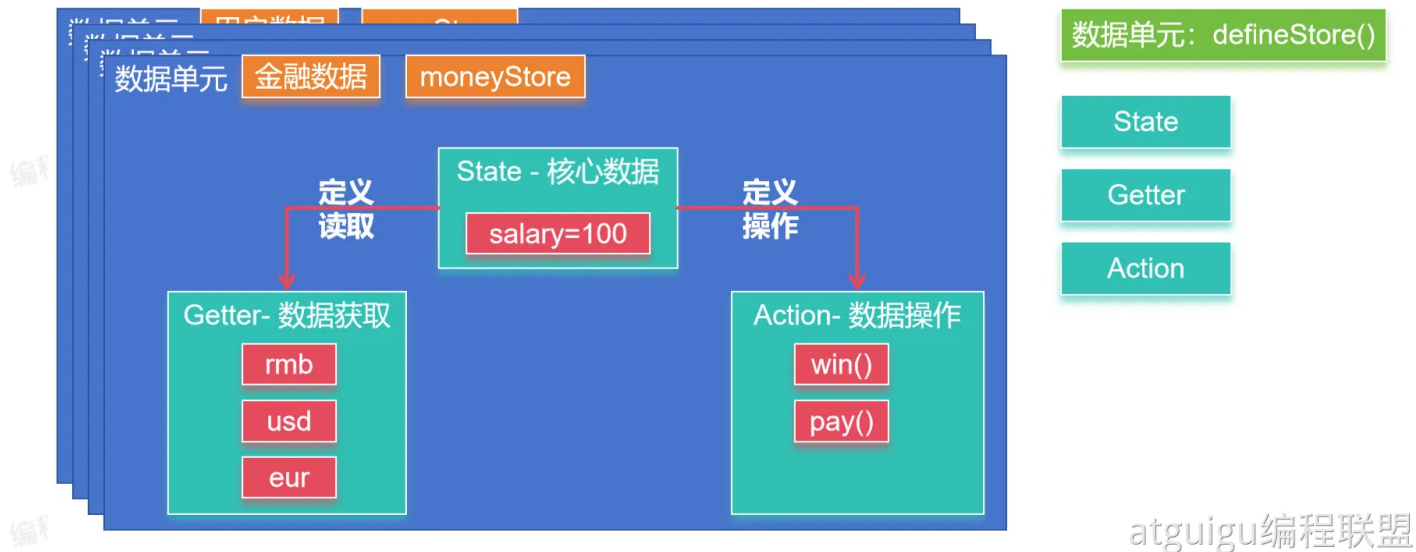
7.1. 核心

Pinia 是 Vue 的存储库，它允许您跨组件/页面共享状态。

Pinia 三个核心概念：

- State：表示 Pinia Store 内部保存的数据（data）
- Getter：可以认为是 Store 里面数据的计算属性（computed）
- Actions：是暴露修改数据的几种方式。

虽然外部也可以直接读写 Pinia Store 中保存的 data，但是我们建议使用 Actions 暴露的方法操作数据更加安全。



7.2. 整合

```
1 npm install pinia
```

main.js

```
1 import { createApp } from 'vue'
2 import './style.css'
3 import App from './App.vue'
4 import { createPinia } from 'pinia'
5
6 const pinia = createPinia();
7
8 createApp(App)
9   .use(pinia)
10  .mount('#app')
```

7.3. 案例

stores/money.js 编写内容;

```
1 import {defineStore} from 'pinia'
2
3 //定义一个 money存储单元
4 export const useMoneyStore = defineStore('money', {
5   state: () => ({money: 100}),
6   getters: {
7     rmb: (state) => state.money,
8     usd: (state) => state.money * 0.14,
9     eur: (state) => state.money * 0.13,
10  },
11  actions: {
12    win(arg) {
13      this.money+=arg;
14    },
15    pay(arg){
16      this.money -= arg;
17    }
18  },
19 });
20
```

App.vue

```
1 <script setup>
2   import Wallet from "./components/Wallet.vue";
3   import Game from "./components/Game.vue";
4 </script>
5
6 <template>
7   <Wallet></Wallet>
8   <Game/>
9
10 </template>
11
12 <style scoped>
13
14 </style>
15
```

Wallet.vue

```
1 <script setup>
2 import {useMoneyStore} from '../stores/money.js'
3 let moneyStore = useMoneyStore();
4
5 </script>
6
7 <template>
8 <div>
9   <h2>¥: {{moneyStore.rmb}}</h2>
10  <h2>$: {{moneyStore.usd}}</h2>
11  <h2>€: {{moneyStore.eur}}</h2>
12 </div>
13 </template>
14
15 <style scoped>
16 div {
17   background-color: #f9f9f9;
18 }
19 </style>
20
```

Game.vue

编程联盟(45022700)

编程联盟(45022700)

编程联盟(45022700)

编程联盟(45022700)

```
1 <script setup>
2 import {useMoneyStore} from '../stores/money.js'
3
4 let moneyStore = useMoneyStore();
5 function guaguale(){
6   moneyStore.win(100);
7 }
8
9 function bangbang(){
10   moneyStore.pay(5)
11 }
12 </script>
13
14 <template>
15   <button @click="guaguale">刮刮乐</button>
16   <button @click="bangbang">买棒棒糖</button>
17 </template>
18
19 <style scoped>
20
21 </style>
22
```

7.4. setup写法

```

1 export const useMoneyStore = defineStore('money', () => {
2   const salary = ref(1000); // ref() 就是 state 属性
3   const dollar = computed(() => salary.value * 0.14); // computed() 就
   是 getters
4   const eur = computed(() => salary.value * 0.13); // computed() 就是 ge
   tters
5
6   //function() 就是 actions
7   const pay = () => {
8     salary.value -= 100;
9   }
10
11   const win = () => {
12     salary.value += 1000;
13   }
14
15   //重要: 返回可用对象
16   return {salary,dollar,eur,pay,win}
17 })

```

8. 脚手架

```

1 npm create vite # 选择 使用 create-vue 自定义项目
2 npm create vue@latest # 直接使用create-vue 创建项目
3 vue-cli #已经过时

```

9. Ant Design Vue

官网: <https://www.antdv.com/>

使用 `npm create vue@latest` 创建出项目脚手架, 然后整合 `ant design vue`

9.1. 整合

安装依赖

```
1 npm i --save ant-design-vue@4.x
```

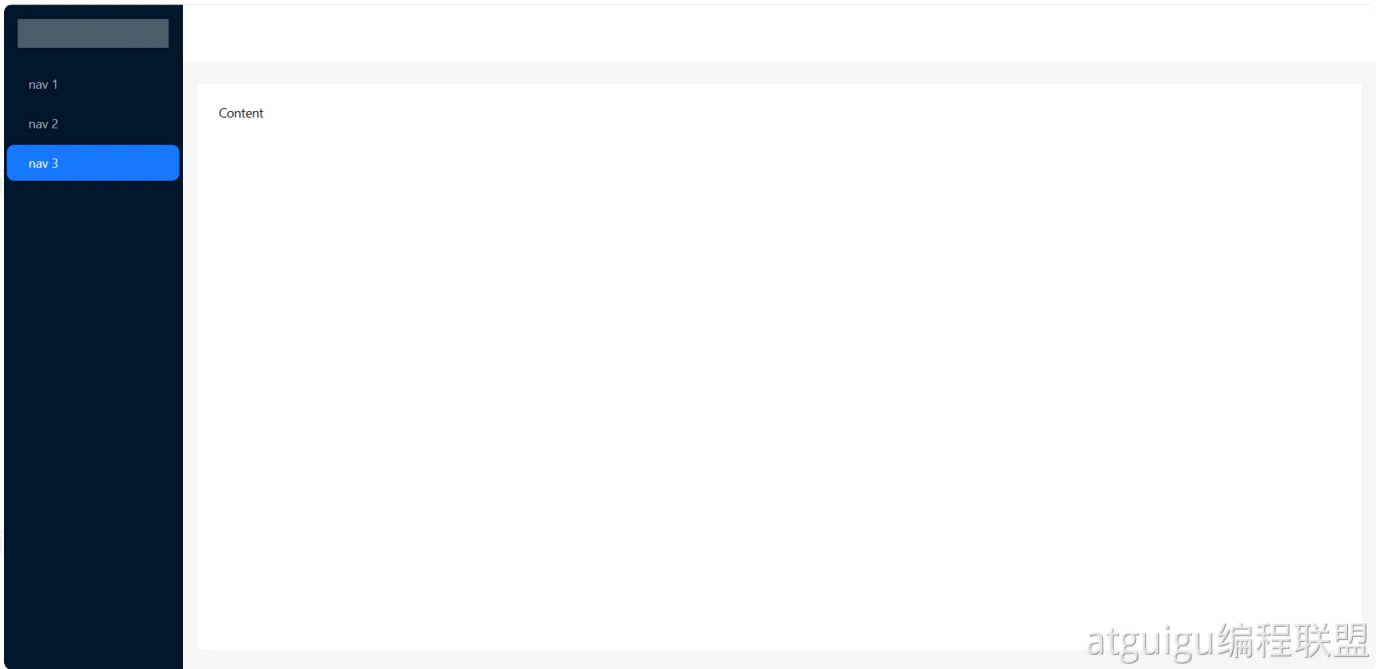
全局注册：编写 `main.js`

```
1 import { createApp } from 'vue';
2 import Antd from 'ant-design-vue';
3 import App from './App';
4 import 'ant-design-vue/dist/reset.css';
5
6 const app = createApp(App);
7
8 app.use(Antd).mount('#app');
```

9.2. 布局

- **Layout**：布局容器，其下可嵌套 **Header Sider Content Footer** 或 **Layout** 本身，可以放在任何父容器中。
 - **Header**：顶部布局，**自带默认样式**，其下可嵌套任何元素，只能放在 **Layout** 中。
 - **Sider**：侧边栏，**自带默认样式及基本功能**，其下可嵌套任何元素，只能放在 **Layout** 中。
 - **Content**：内容部分，**自带默认样式**，其下可嵌套任何元素，只能放在 **Layout** 中。
 - **Footer**：底部布局，**自带默认样式**，其下可嵌套任何元素，只能放在 **Layout** 中。

一个典型的后台管理系统布局



编程联盟(45022700)

编程联盟(45022700)

编程联盟(45022700)

编程联盟(45022700)

编程联盟(45022700)

编程联盟(45022700)

```
1 <template>
2   <a-layout id="components-layout-demo" style="height: 100vh">
3     <a-layout-sider v-model:collapsed="collapsed" :trigger="null" collapsi
4       ble>
5       <div class="logo" />
6       <a-menu v-model:selectedKeys="selectedKeys" theme="dark" mode="inlin
7         e">
8         <a-menu-item key="1">
9           <user-outlined />
10          <span>nav 1</span>
11        </a-menu-item>
12        <a-menu-item key="2">
13          <video-camera-outlined />
14          <span>nav 2</span>
15        </a-menu-item>
16        <a-menu-item key="3">
17          <upload-outlined />
18          <span>nav 3</span>
19        </a-menu-item>
20      </a-menu>
21    </a-layout-sider>
22    <a-layout>
23      <a-layout-header style="background: #fff; padding: 0">
24        <menu-unfold-outlined
25          v-if="collapsed"
26          class="trigger"
27          @click="() => (collapsed = !collapsed)"
28        />
29        <menu-fold-outlined v-else class="trigger" @click="() => (collapse
30          d = !collapsed)" />
31      </a-layout-header>
32      <a-layout-content
33        :style="{ margin: '24px 16px', padding: '24px', background: '#fff', minHeight: '280px' }"
34      >
35        Content
36      </a-layout-content>
37    </a-layout>
38  </template>
39  <script setup>
40    import { ref } from 'vue';
41    const selectedKeys = ref(['1']);
42    const collapsed = ref(false);
43  </script>
```

```

42 <style>
43 #components-layout-demo .trigger {
44   font-size: 18px;
45   line-height: 64px;
46   padding: 0 24px;
47   cursor: pointer;
48   transition: color 0.3s;
49 }
50
51 #components-layout-demo .trigger:hover {
52   color: #1890ff;
53 }
54
55 #components-layout-demo .logo {
56   height: 32px;
57   background: rgba(255, 255, 255, 0.3);
58   margin: 16px;
59 }
60
61 .site-layout .site-layout-background {
62   background: #fff;
63 }
64 </style>
65

```

9.3. 组件

参照官网使用即可

10. 接下来

做一个项目的后台管理系统，完全打通前端工程化的使用即可；

推荐：

- 《谷粒商城 2024 – 全栈篇》

